

SmartOven

Si vuole modellare in TypeScript un forno programmabile.

Implementare una classe **SmartOven** con:

- **serial** (un numero di serie, stringa, non modificabile);
- **temperature** (numero intero);
- **mode** (una modalità di funzionamento);
- **door_open** (stato della porta, booleano, aperta/chiusa).

Le modalità disponibili sono: `Off`, `Static`, `Fan`, `Grill`, da rappresentare tramite una enum **OvenMode**.

Il forno deve essere inizializzato con:

- modalità spenta;
- temperatura pari a 0;
- porta chiusa.

Devono essere disponibili operazioni per:

- leggere le informazioni del forno (*serial*, *temperatura*, *mode*, *door_open*);
- modificare la temperatura;
- modificare la modalità (tramite un metodo **setMode(newMode)**);
- aprire e chiudere la porta (tramite i metodi **openDoor**, **closeDoor**).

Devono valere i seguenti vincoli:

- se il forno è spento, la temperatura deve essere sempre 0;
- non è possibile impostare una temperatura diversa da 0 quando il forno è spento;
- non è possibile impostare la modalità Grill se la porta è aperta;
- non è possibile aprire la porta se la temperatura è maggiore di 150.

In caso di operazione non valida deve essere sollevata un'eccezione

InvalidOvenOperationError, che contenga almeno il nome dell'operazione che ha causato l'errore (es: "openDoor", "setMode", "temperature") in un attributo *operation*. Infine, si definisca una funzione **activeOvens(ovens)** che, dato un array di forni, restituisca il numero di forni attivi (cioè non spenti).

Si curi l'annotazione dei tipi nel modo più preciso possibile.

Esempio d'uso:

```
const a = new SmartOven("A1")
const b = new SmartOven("B2")
```

```
a.setMode(OvenMode.Static)
b.setMode(OvenMode.Off)
```

```
activeOvens([a, b]) // 1
```

```
1  class InvalidOvenOperationError extends Error {
2      public operation: string
3
4      constructor(message: string, operation: string) {
5          super(message)
6          this.operation = operation
7      }
8  }
9
10 enum OvenMode {
11     Off = 0,
12     Static = 1,
13     Fan = 2,
14     Grill = 3
15 }
16
17 class SmartOven {
18     private _serial: string
19     private _temperature: number
20     private _mode: OvenMode
21     private _door_open: boolean
22
23     constructor(serial: string) {
24         this._serial = serial
25         this._temperature = 0
26         this._mode = OvenMode.Off
27         this._door_open = false
28     }
29
30     get serial(): string { return this._serial }
31     get temperature(): number { return this._temperature }
32     get mode(): OvenMode { return this._mode }
33     get door_open(): boolean { return this._door_open }
34
35     set temperature(value: number) {
36         if (this._mode == OvenMode.Off && value != 0)
37             throw new InvalidOvenOperationError("temperature not allowed in off mode", "temperature")
38         this._temperature = value
39     }
40
41     setMode(new_mode: OvenMode): void {
42         if (new_mode == OvenMode.Grill && this._door_open)
43             throw new InvalidOvenOperationError("grill not allowed with open door", "setMode")
44         this._mode = new_mode
45         if (this._mode == OvenMode.Off)
46             this._temperature = 0
47     }
48
49     openDoor(): void {
50         if (this._temperature > 150)
51             throw new InvalidOvenOperationError("door cannot be opened", "openDoor")
52         this._door_open = true
53     }
54
55     closeDoor(): void {
56         this._door_open = false
57     }
58 }
59
60 function activeOvens(ovens: SmartOven[]): number {
61     let c: number = 0
62     for (let oven of ovens)
63         if (oven.mode != OvenMode.Off) c++
64     return c
65 }
```

UndoableHistory

Si vuole modellare in TypeScript una struttura che mantenga la storia di operazioni applicate a un valore.

Ogni operazione è composta da:

- una funzione **apply** che trasforma un valore in un altro valore dello stesso tipo;
- una funzione **undo** che annulla l'effetto della trasformazione;
- una descrizione **description** (*opzionale*).

Scrivere una classe **UndoableHistory** che:

- parte da un valore iniziale.
- permettere di aggiungere nuove operazioni: **add(apply_fn, undo_fn, desc)**;
- permettere di ottenere il valore corrente *current_value* applicando tutte le operazioni in ordine;
- permettere di annullare l'ultima operazione inserita: **undoLast()**.

Se si tenta di annullare un'operazione quando non ce ne sono, deve essere sollevata un'eccezione **EmptyHistoryError**.

Si definisca inoltre una funzione **replay(initial_value, operations)** che, dato un valore iniziale e una lista di operazioni, restituisca il valore finale ottenuto applicandole in ordine.

Si curi l'annotazione dei tipi nel modo più preciso possibile.

Esempio d'uso:

```
const h = new UndoableHistory(10)
```

```
h.add(x ⇒ x + 5, x ⇒ x - 5)
```

```
h.add(x ⇒ x * 2, x ⇒ x / 2)
```

```
h.current_value // 30
```

```
h.undoLast()
```

```
h.current_value // 15
```

```
1  class EmptyHistoryError extends Error { ; }
2
3  type OperationFunction<I> = (x: I) => I
4
5  interface Operation<I> {
6      apply: OperationFunction<I>,
7      undo: OperationFunction<I>,
8      description?: string
9  }
10
11 class UndoableHistory<I> {
12     public initial_value: I
13     public operations: Operation<I>[]
14
15     constructor(initial_value: I) {
16         this.initial_value = initial_value
17         this.operations = []
18     }
19
20     get current_value(): I {
21         let c_val: I = this.initial_value
22         for (let op of this.operations)
23             c_val = op.apply(c_val)
24         return c_val
25     }
26
27     add(apply_function: OperationFunction<I>,
28         undo_function: OperationFunction<I>,
29         description?: string): void {
30         this.operations.push({
31             apply: apply_function,
32             undo: undo_function,
33             description: description
34         })
35     }
36
37     undoLast(): void {
38         if (this.operations.length == 0)
39             throw new EmptyHistoryError("empty history")
40         this.operations.pop()
41     }
42 }
43
44 function replay<I>(initial_value: I, operations: Operation<I>[]): I {
45     let c_val: I = initial_value
46     for (let op of operations)
47         c_val = op.apply(c_val)
48     return c_val
49 }
```

GroupBy

Si vuole implementare in TypeScript una funzione generica che raggruppi gli elementi di un array in base a una chiave testuale.

Si definisca una funzione **groupBy(items)** che riceve un array di elementi dello stesso tipo, ciascuno con almeno un campo *key* (string).

La funzione deve restituire un oggetto che associa a ogni valore di *key*, l'array degli elementi che hanno quella chiave.

Si definisca poi una funzione **groupUniqueByName(items)** che:

- costruisca i gruppi come nella funzione precedente, usando il campo *key*;
- all'interno di ogni gruppo, eviti di inserire elementi con lo stesso valore del campo *name* (string);
- mantenga solo la prima occorrenza incontrata (rispetto al campo *name*).

Infine, si scriva una funzione **countGroups(groups)** che, dato un oggetto prodotto da *groupBy* o *groupUniqueByName*, restituisca il numero di chiavi presenti.

Si curi l'annotazione dei tipi nel modo più preciso possibile.

Esempio:

```
const items = [  
  { key: "rome", name: "Anna", surname: "Rossi" },  
  { key: "rome", name: "Luca", surname: "Verdi" },  
  { key: "rome", name: "Anna", surname: "Verdi" },  
  { key: "milan", name: "Anna", surname: "Bianchi" }  
]
```

```
const g = groupBy(items)  
g["rome"].length // 3
```

```
const u = groupUniqueByName(items)  
u["rome"].length // 2
```

```
countGroups(u) // 2
```

```
1  interface KeyedItem {
2      key: string
3  }
4
5  interface NamedItem {
6      key: string,
7      name: string
8  }
9
10 function groupBy<I extends KeyedItem>(items: I[]): { [key: string]: I[] } {
11     let groups: { [key: string]: I[] } = {}
12
13     for (let item of items) {
14         if (groups[item.key] == undefined)
15             groups[item.key] = []
16         groups[item.key].push(item)
17     }
18
19     return groups
20 }
21
22 function groupUniqueByName<I extends NamedItem>(items: I[]): { [key: string]: I[] } {
23     let groups: { [key: string]: I[] } = {}
24
25     for (let item of items) {
26         if (groups[item.key] == undefined)
27             groups[item.key] = []
28
29         let found: boolean = false
30         for (let other of groups[item.key]) {
31             if (other.name == item.name) {
32                 found = true
33                 break
34             }
35         }
36
37         if (!found)
38             groups[item.key].push(item)
39     }
40
41     return groups
42 }
43
44 function countGroups<I extends KeyedItem>(groups: { [key: string]: I[] }): number {
45     let c: number = 0
46     for (let k in groups)
47         c++
48     return c
49 }
```